AI Global Architecture

Diagrams & Component Definitions

Multi-Agent System | AI Engines | Communication Protocols

Senior AI Engineer Architecture Document

Following Best Practices & Design Patterns

Table of Contents

1. AI Global Architecture Overview

2. Multi-Agent Communication Architecture

3. Agent Details:

3.1 Interview Agent

3.2 Scoring Agent

3.3 Avatar Agent

3.4 Context Agent (RAG)

4. AI Engines:

4.1 NLP Pipeline

4.2 Computer Vision Pipeline

4.3 ASR Engine

4.4 TTS Engine

4.5 NER Engine

4.6 Emotion Detection

5. Communication Protocols

6. AI Design Patterns

1. AI Global Architecture Overview

The AI Global Architecture represents the complete intelligent system powering the AI Recruiter platform. This architecture follows a modular, microservices-based design where specialized AI components operate as independent services orchestrated by a central coordination layer. The system implements the separation of concerns principle, ensuring each component has a well-defined responsibility while enabling seamless integration through standardized communication protocols.

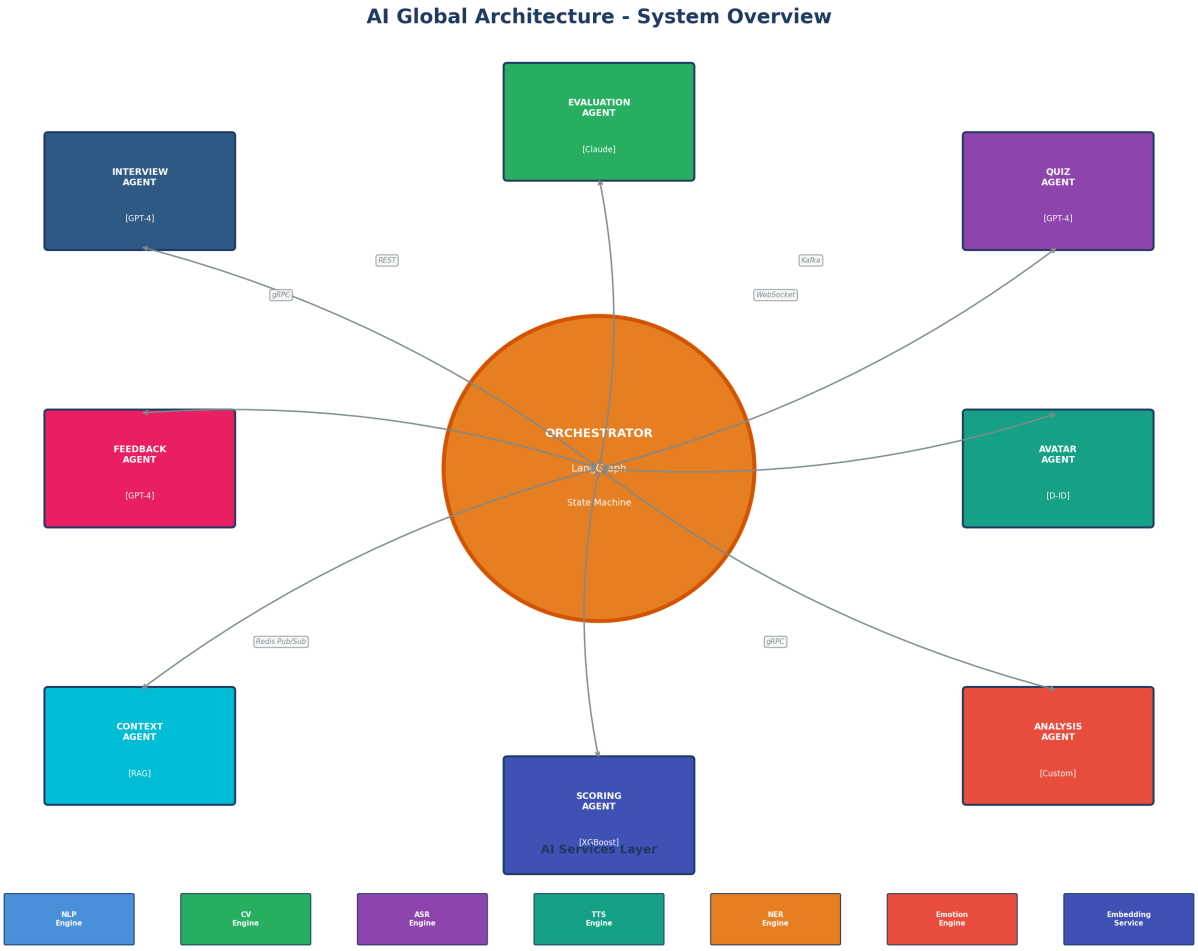

Figure 1: AI Global Architecture - System Overview

1.1 Core Components

The architecture consists of multiple layers of intelligence, from high-level agent coordination to low-level AI model inference. The central Orchestrator manages conversation state and coordinates all agent interactions through a state machine implemented with LangGraph. Surrounding the orchestrator are specialized agents, each designed for specific tasks in the interview workflow.

Component	Role	Technology	Responsibility
Orchestrator	Central Hub	LangGraph	State management, workflow coordination
Interview Agent	Conversation	GPT-4, RAG	Question generation, dialogue management
Evaluation Agent	Assessment	Claude	Response quality evaluation
Quiz Agent	Testing	GPT-4	Technical question generation
Avatar Agent	Presentation	D-ID, ElevenLabs	Video/speech synthesis
Analysis Agent	Processing	Custom Models	Multi-modal signal analysis

Scoring Agent	Decision	XGBoost	Score aggregation, recommendations
Context Agent	Knowledge	RAG, Pinecone	Context retrieval, memory
Feedback Agent	Reporting	GPT-4	Report generation, summaries

Table 1: AI Architecture Core Components

2. Multi-Agent Communication Architecture

The multi-agent communication architecture implements an event-driven model where agents interact through a centralized message bus. This design ensures loose coupling between agents while maintaining reliable message delivery and enabling horizontal scaling. Each agent operates independently, processing messages according to its capabilities and publishing results back to the system.

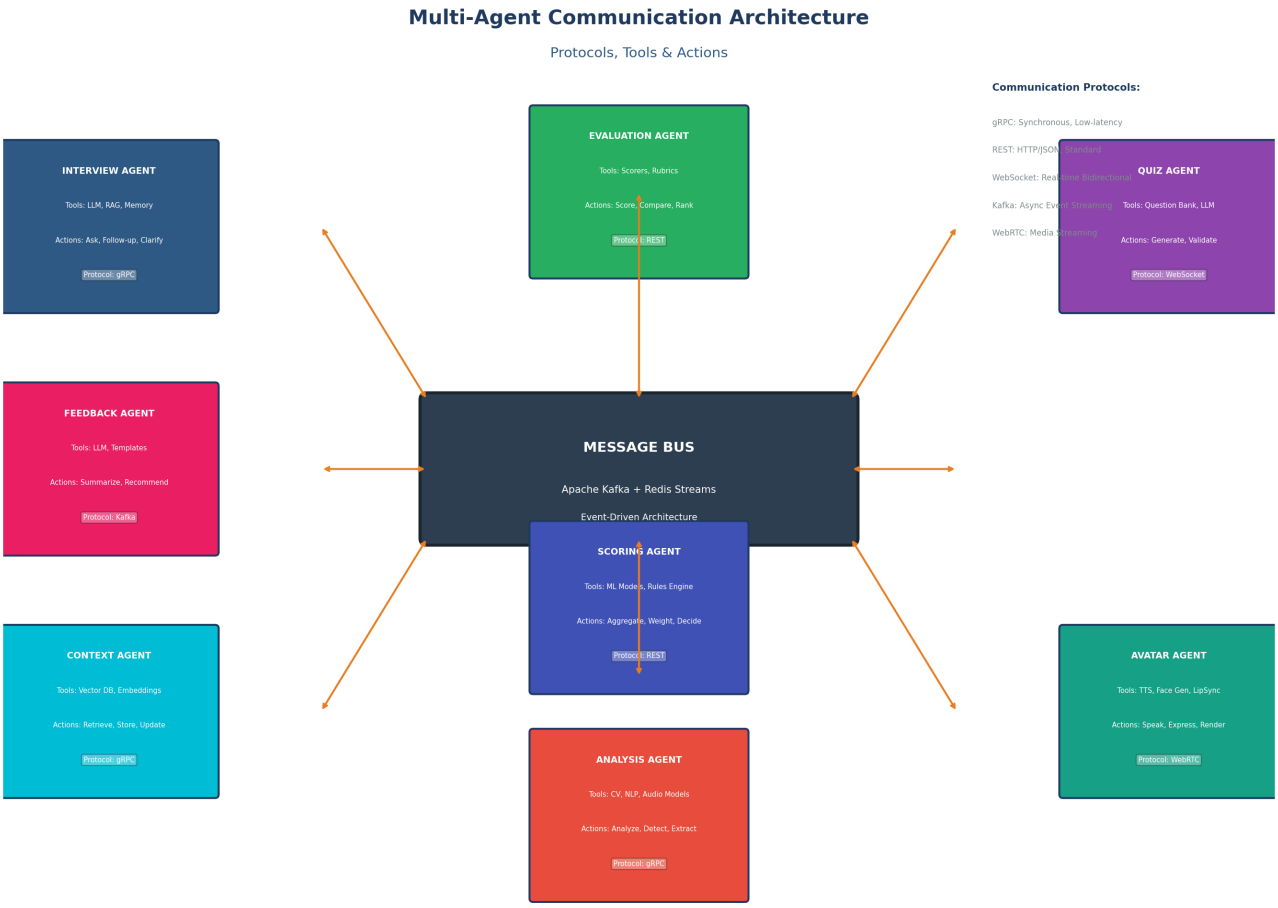

Figure 2: Multi-Agent Communication Architecture

2.1 Message Bus Architecture

The message bus serves as the communication backbone, implemented using Apache Kafka for persistent event streaming and Redis Streams for real-time, low-latency messaging. This dual-bus architecture provides both durability for critical events and speed for time-sensitive operations. The event-driven pattern enables asynchronous processing, fault isolation, and natural load balancing across agent instances.

2.2 Communication Protocols

Protocol	Use Case	Latency	Pattern
gRPC	AI model inference, real-time analysis	10-50ms	Request-Response
REST	CRUD operations, configuration	50-200ms	Request-Response
WebSocket	Live interview streaming	20-100ms	Bidirectional
Kafka	Event logging, async processing	5-500ms	Pub/Sub
WebRTC	Video/audio capture	100-300ms	Peer-to-Peer

Table 2: Communication Protocols Comparison

3. Agent Details

3.1 Interview Agent

The Interview Agent is the primary conversational interface between the AI system and the candidate. It leverages a Large Language Model (GPT-4 or Claude) integrated with Retrieval-Augmented Generation (RAG) to conduct contextual, adaptive interviews. The agent maintains conversation state, generates relevant follow-up questions, and adapts its questioning strategy based on candidate responses.

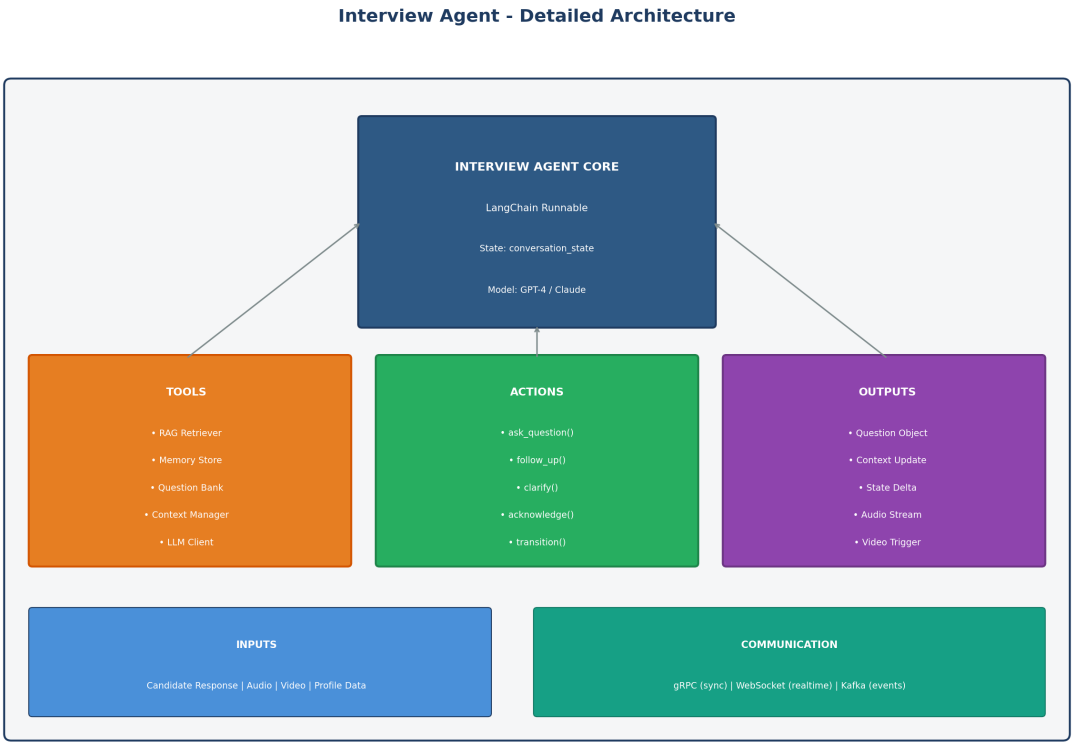

Figure 3: Interview Agent - Detailed Architecture

Tools & Capabilities

- RAG Retriever: Retrieves relevant context from knowledge base for question generation
- Memory Store: Maintains conversation history and candidate responses
- Question Bank: Database of role-specific questions organized by domain
- Context Manager: Tracks interview state and adapts questioning flow
- LLM Client: Interface to GPT-4/Claude for natural language generation

Actions

- ask_question(): Generates and presents the next interview question
- follow_up(): Creates contextual follow-up based on candidate response
- clarify(): Requests clarification when response is ambiguous
- acknowledge(): Acknowledges candidate response and provides feedback
- transition(): Moves to the next interview phase or topic

3.2 Scoring Agent

The Scoring Agent aggregates signals from all evaluation modules into a unified candidate assessment. It implements a weighted ensemble approach combining machine learning predictions with rule-based scoring logic. The agent produces both numerical scores and qualitative recommendations that HR professionals use for decision-making.

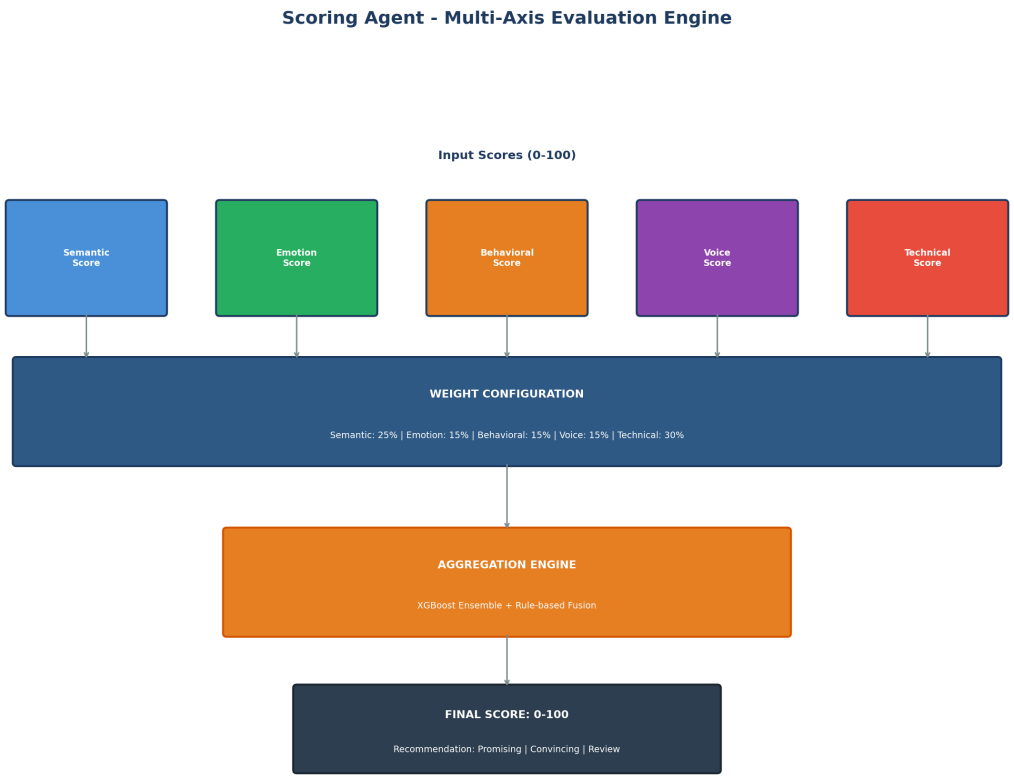

Figure 4: Scoring Agent - Multi-Axis Evaluation Engine

Scoring Weights

Evaluation Axis	Weight	Source	Range
Semantic Score	25%	NLP Module	0-100
Emotion Score	15%	Emotion Detection	0-100
Behavioral Score	15%	CV Module	0-100
Voice Score	15%	Voice Analysis	0-100
Technical Score	30%	Quiz + NLP	0-100

Table 3: Scoring Weights Configuration

3.3 Avatar Agent

The Avatar Agent generates a realistic AI interviewer with synchronized speech, facial expressions, and lip movements. It creates an engaging interview experience by rendering a human-like avatar that conducts natural conversations. The agent integrates multiple synthesis technologies to achieve real-time or near-real-time video output with minimal latency.

Figure 5: Avatar Agent - Real-time Generation Pipeline

Tools & APIs

- ElevenLabs TTS: Neural text-to-speech with emotion control and natural prosody
- D-ID / SadTalker: Face generation and animation from audio input
- Wav2Lip: Lip synchronization model for realistic speech animation
- GPT-4: Response generation for natural conversation flow
- WebRTC: Real-time video streaming to client applications

Performance Targets

Latency: < 500ms end-to-end | Resolution: 720p | FPS: 30 | Codec: H.264

3.4 Context Agent (RAG)

The Context Agent implements Retrieval-Augmented Generation (RAG) to provide the interview system with relevant domain knowledge, company information, and role-specific context. It manages a vector database of documents and enables semantic search to augment LLM prompts with retrieved context, improving the relevance and accuracy of generated questions and responses.

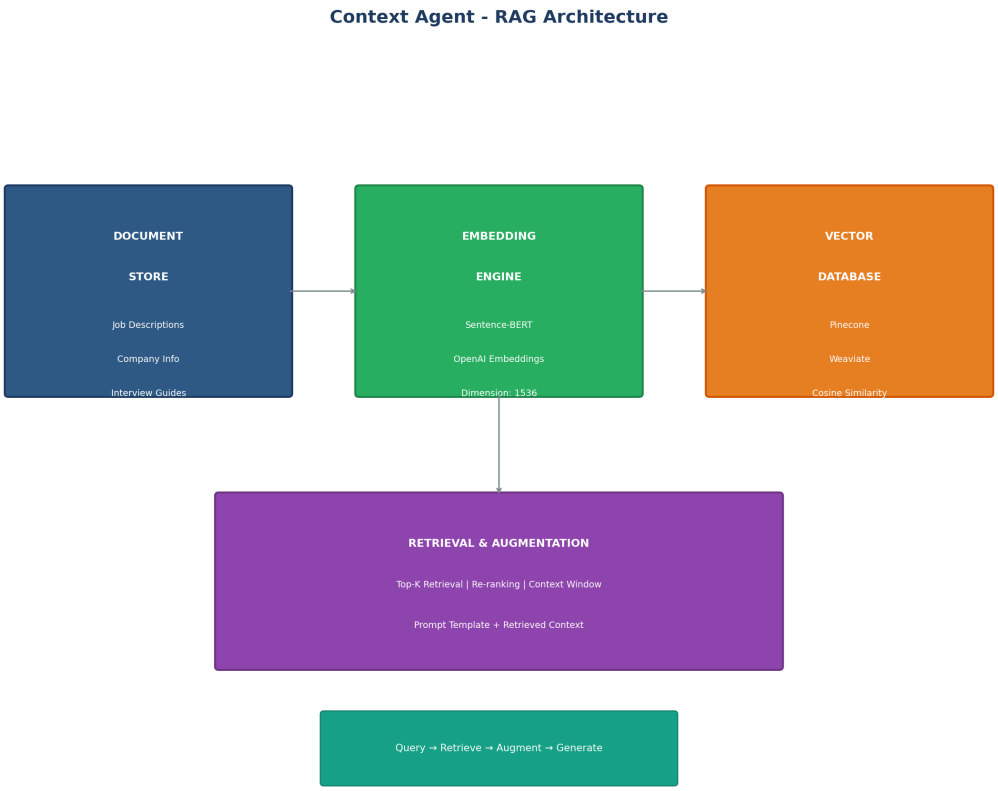

Figure 6: Context Agent - RAG Architecture

RAG Components

- Document Store: MongoDB storing job descriptions, company info, interview guides
- Embedding Engine: Sentence-BERT / OpenAI Embeddings (1536 dimensions)
- Vector Database: Pinecone / Weaviate for semantic similarity search
- Retriever: Top-K retrieval with cosine similarity threshold > 0.75
- Re-ranker: Cross-encoder model for result re-ranking

4. AI Engines

4.1 NLP Engine

The NLP Engine processes all text-based inputs including candidate responses, transcribed speech, and document content. It implements a multi-stage pipeline from raw text input through feature extraction, analysis, and structured output generation. The engine employs multiple transformer-based models optimized for different NLP tasks.

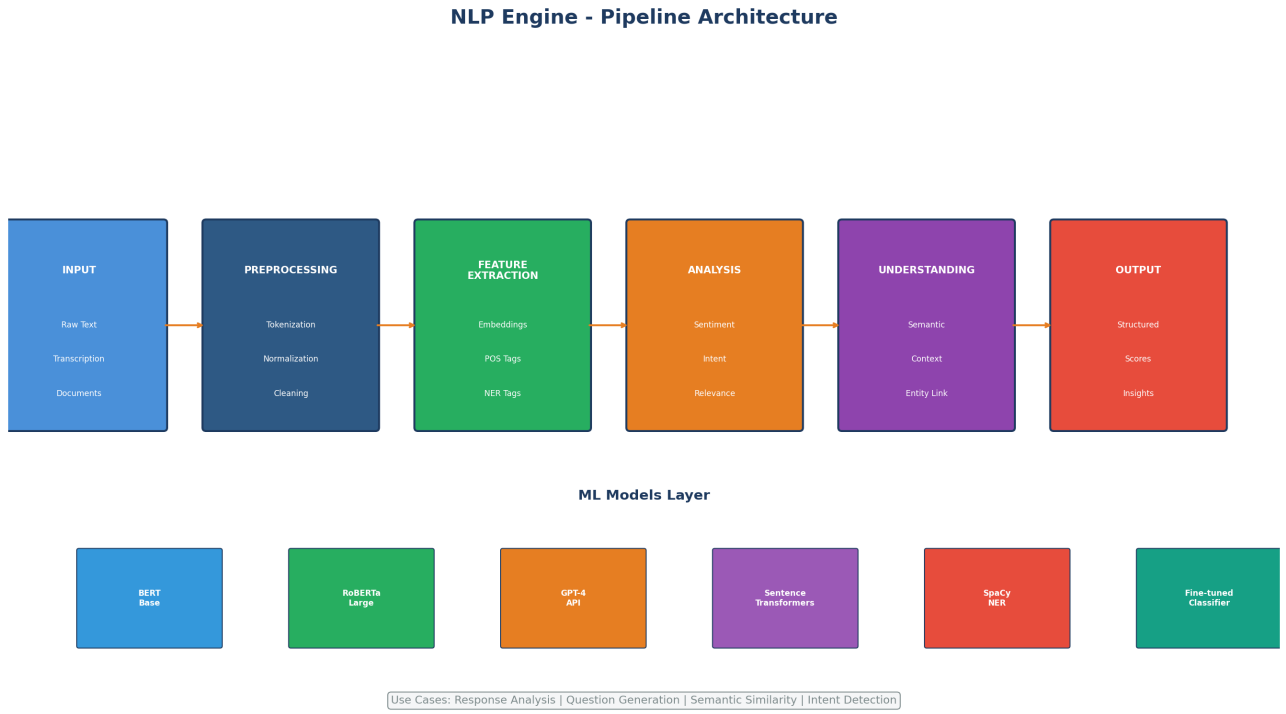

Figure 7: NLP Engine - Pipeline Architecture

Pipeline Stages

- Input: Raw text from transcriptions, documents, or direct input
- Preprocessing: Tokenization, normalization, cleaning, language detection
- Feature Extraction: Embeddings (BERT), POS tagging, NER tagging
- Analysis: Sentiment analysis, intent detection, relevance scoring
- Understanding: Semantic parsing, context integration, entity linking
- Output: Structured data with scores, entities, and insights

Models Deployed

- BERT Base: General-purpose embeddings and feature extraction
- RoBERTa Large: Advanced text understanding and classification
- GPT-4 API: Natural language generation and complex reasoning
- Sentence Transformers: Semantic similarity and embedding generation
- SpaCy NER: Named entity recognition and extraction
- Fine-tuned Classifier: Custom model for interview-specific tasks

4.2 Computer Vision Engine

The Computer Vision Engine analyzes video streams from the interview session to extract behavioral signals. It processes frames in real-time to detect faces, recognize emotions, track gaze direction, and estimate body pose. These signals contribute to the behavioral assessment component of candidate evaluation.

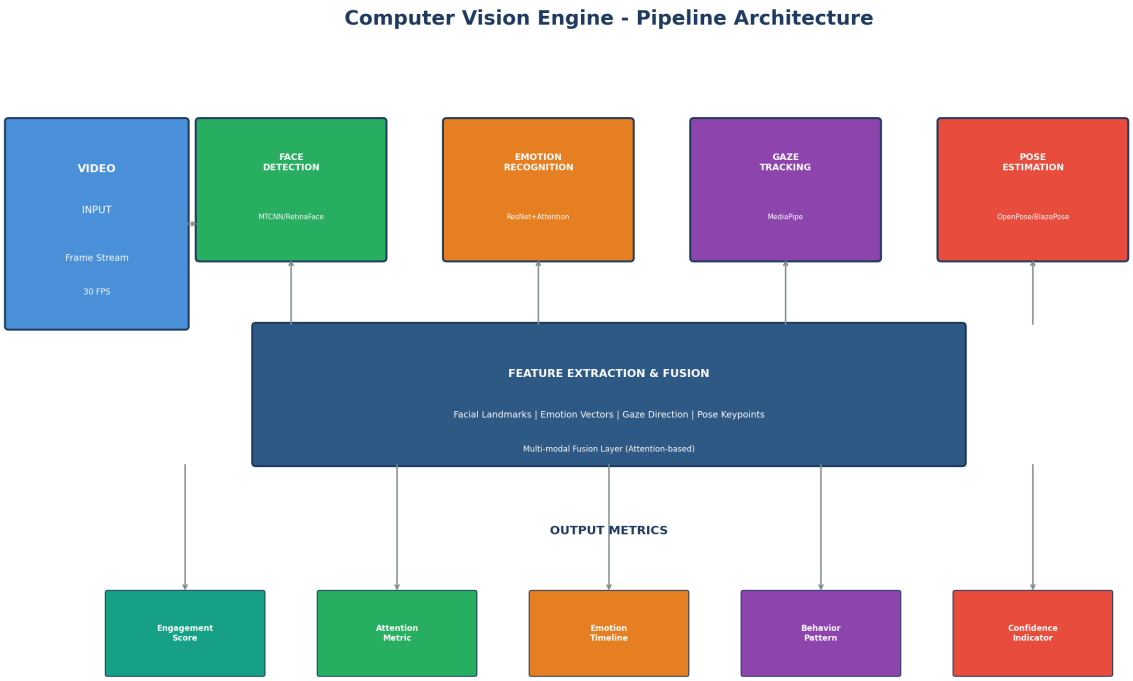

Figure 8: Computer Vision Engine - Pipeline Architecture

Processing Branches

- Face Detection: MTCNN/RetinaFace for robust face localization at 30 FPS
- Emotion Recognition: ResNet with attention mechanism for 7 basic emotions
- Gaze Tracking: MediaPipe for eye tracking and attention direction
- Pose Estimation: OpenPose/BlazePose for body language analysis

Output Metrics

Engagement Score | Attention Metric | Emotion Timeline | Behavior Pattern | Confidence Indicator

4.3 ASR Engine

The ASR (Automatic Speech Recognition) Engine converts candidate speech to text in real-time. It implements a streaming architecture that processes audio chunks as they arrive, enabling live transcription with minimal latency. The engine supports multiple languages and includes features like speaker diarization and automatic punctuation.

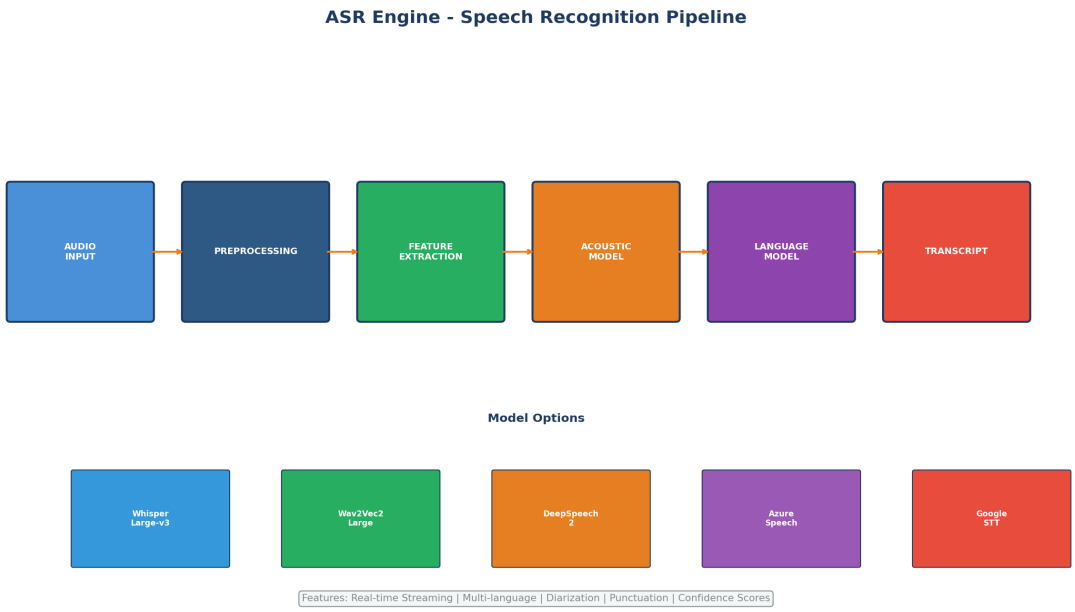

Figure 9: ASR Engine - Speech Recognition Pipeline

Pipeline Stages

- Audio Input: Real-time audio stream from WebRTC (16kHz, mono)
- Preprocessing: Noise reduction, VAD, audio normalization
- Feature Extraction: Mel-spectrogram or filter bank features
- Acoustic Model: Neural network for audio-to-phoneme mapping
- Language Model: Statistical model for text prediction
- Transcript: Final text output with confidence scores

Model Options

Whisper Large-v3 | Wav2Vec2 Large | DeepSpeech 2 | Azure Speech | Google STT

4.4 TTS Engine

The TTS (Text-to-Speech) Engine generates natural-sounding speech from text input for the avatar interviewer. It employs neural synthesis models that produce human-like speech with proper prosody, emotion, and emphasis. The engine supports multiple voices and enables real-time synthesis for interactive conversations.

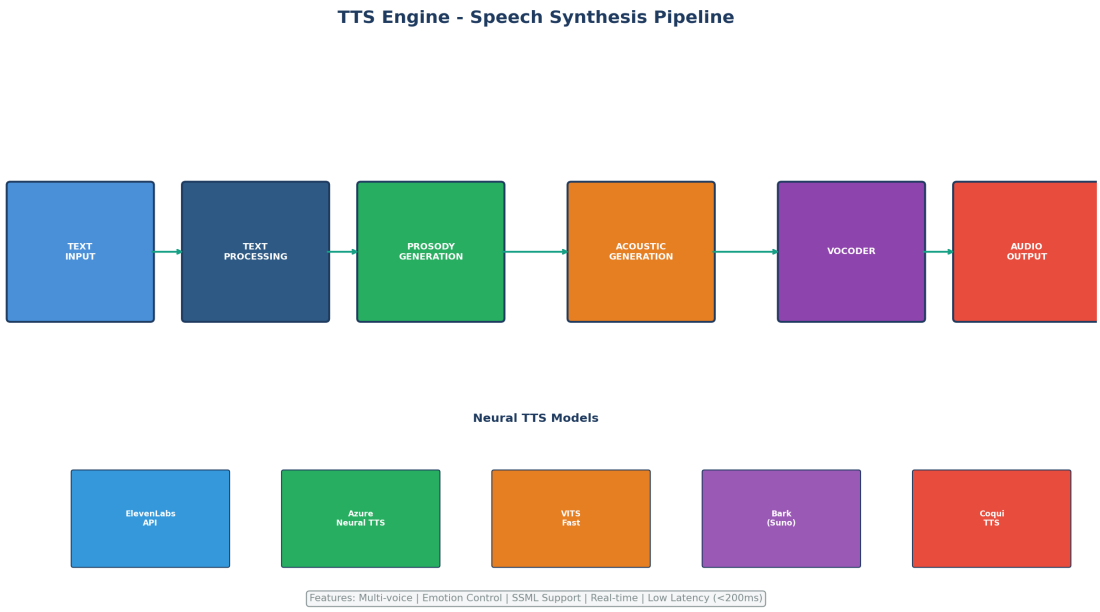

Figure 10: TTS Engine - Speech Synthesis Pipeline

Pipeline Stages

- Text Input: Generated response from Interview Agent
- Text Processing: Normalization, phoneme conversion, SSML parsing
- Prosody Generation: Duration and pitch prediction
- Acoustic Generation: Mel-spectrogram generation from text
- Vocoder: Waveform generation from mel-spectrogram
- Audio Output: PCM audio stream for avatar lip-sync

4.5 NER Engine

The NER (Named Entity Recognition) Engine identifies and classifies named entities in candidate responses and documents. It extracts skills, organizations, dates, people, locations, and educational institutions to build structured candidate profiles and enable semantic search across application data.

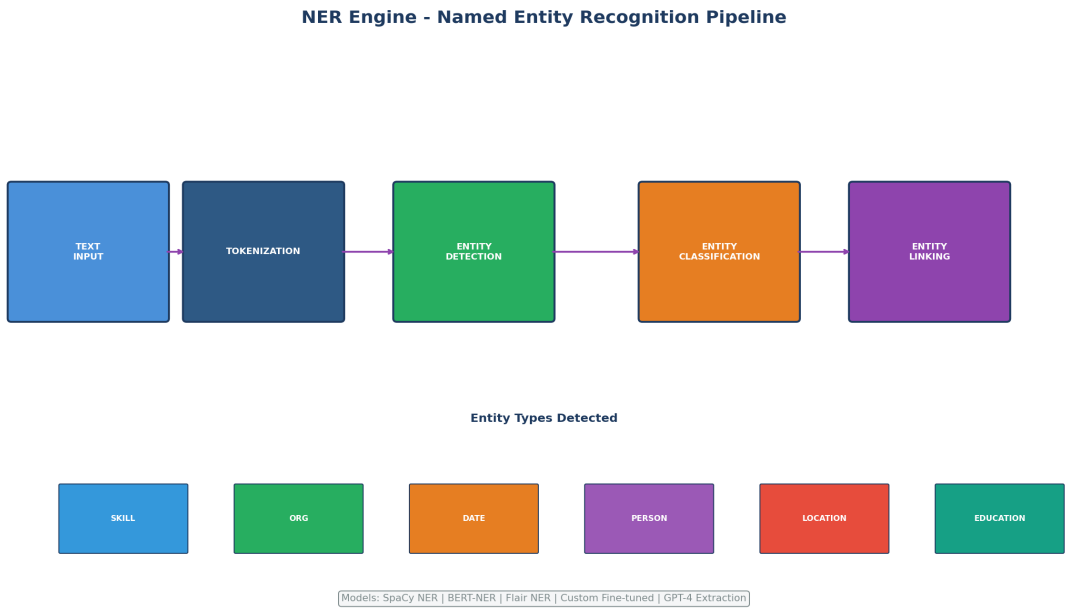

Figure 11: NER Engine - Named Entity Recognition Pipeline

Entity Types

- SKILL: Technical skills, programming languages, frameworks
- ORG: Companies, institutions, organizations
- DATE: Dates, durations, time periods
- PERSON: Names of people mentioned
- LOCATION: Cities, countries, addresses
- EDUCATION: Degrees, universities, certifications

4.6 Emotion Detection Engine

The Emotion Detection Engine implements multi-modal emotion analysis by fusing signals from facial expressions, voice prosody, and text sentiment. It follows the Plutchik model of emotions, detecting eight primary emotional states with intensity scores. The fusion layer uses attention-based cross-modal integration to produce unified emotion predictions.

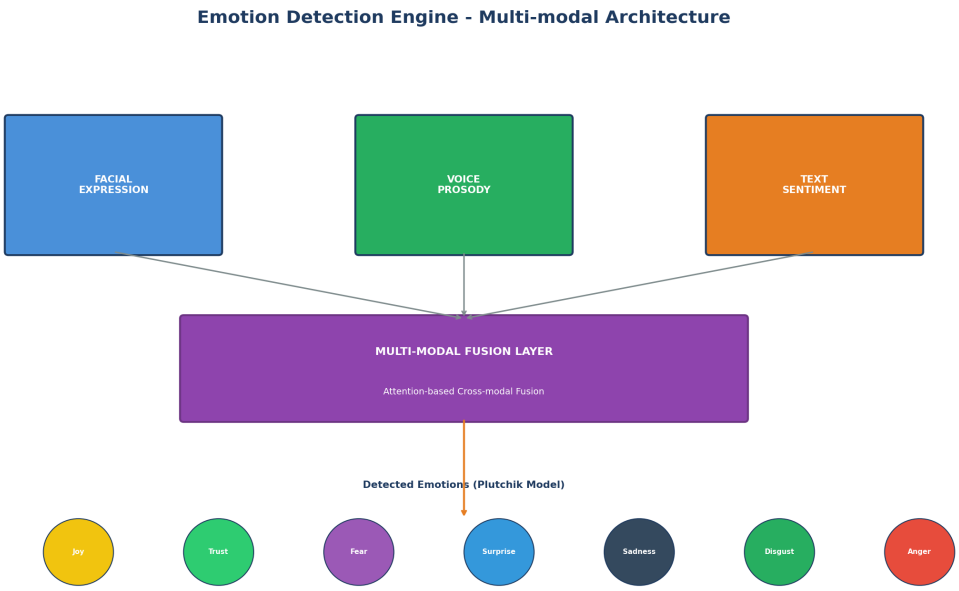

Figure 12: Emotion Detection Engine - Multi-modal Architecture

Input Modalities

- Facial Expression: Frame-by-frame analysis using ResNet variants
- Voice Prosody: Acoustic feature extraction (pitch, energy, rhythm)
- Text Sentiment: Transformer-based sentiment classification

Detected Emotions (Plutchik Model)

Joy | Trust | Fear | Surprise | Sadness | Disgust | Anger | Anticipation

5. Communication Protocols

The system employs multiple communication protocols optimized for different use cases. The selection criteria include latency requirements, reliability needs, data volume, and integration complexity. Each protocol serves specific purposes within the overall architecture.

Figure 13: Communication Protocols Comparison

5.1 Protocol Selection Guide

- gRPC: Use for synchronous AI model inference requiring low latency (< 50ms). Ideal for real-time analysis tasks like emotion detection and scoring.
- REST: Use for standard CRUD operations, configuration management, and external API integration. Simple to implement and debug.
- WebSocket: Use for bidirectional real-time communication during live interviews. Enables instant message delivery in both directions.
- Kafka: Use for event sourcing, audit logging, and asynchronous task processing. Provides durability and replay capability.
- WebRTC: Use for capturing video and audio streams from the candidate's browser. Enables peer-to-peer media streaming.

6. AI Design Patterns

The architecture follows established AI and software design patterns to ensure maintainability, scalability, and reliability. These patterns address common challenges in AI systems including agent coordination, fault tolerance, and knowledge management.

Figure 14: AI Design Patterns - Best Practices

6.1 Applied Patterns

- Agent Pattern: Autonomous agents with tools, memory, and reasoning capabilities. Each agent operates independently while coordinating through the orchestrator.
- RAG Pattern: Retrieval-Augmented Generation for knowledge-intensive tasks. Retrieves relevant context before generation to improve accuracy and relevance.
- Chain of Thought: Step-by-step reasoning for complex evaluation tasks. Enables transparent decision-making and explainable AI outputs.
- Ensemble Pattern: Multiple models combined for robust predictions. Reduces variance and improves generalization through model diversity.
- Circuit Breaker: Fault tolerance for AI service calls. Prevents cascade failures by failing fast when dependent services are unhealthy.
- Saga Pattern: Distributed transaction management across agents. Ensures consistency in multi-step interview workflows.
- CQRS: Command Query Responsibility Segregation for read/write optimization. Separates scoring (write) from reporting (read) workloads.
- Event Sourcing: State changes captured as immutable events. Enables audit trails, replay, and temporal queries.

6.2 Benefits

These patterns collectively ensure: Scalability (horizontal scaling of agents) | Reliability (fault isolation and recovery) | Maintainability (modular, testable components) | Observability (event-based monitoring and tracing) | Explainability (transparent decision processes)